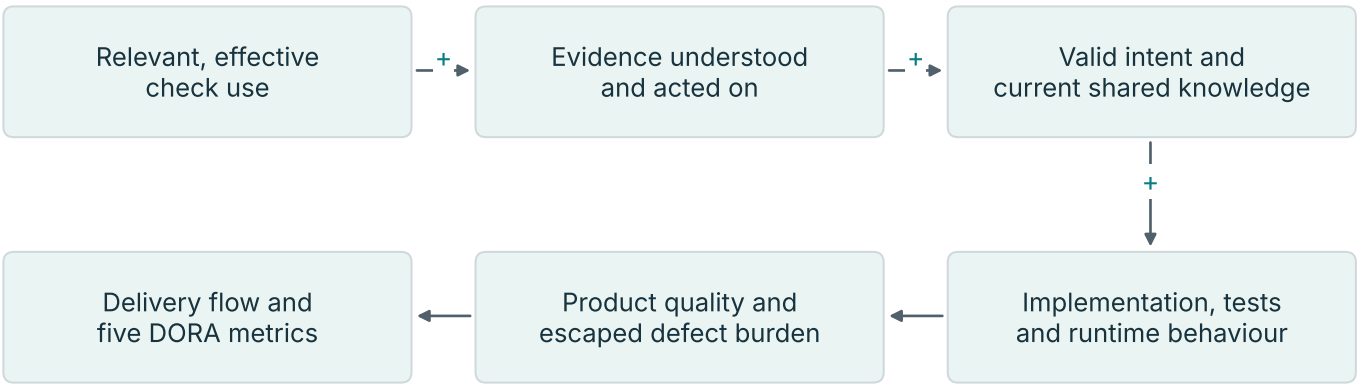
Quality emerges from the system

A causal model of the 20 artefact classes on fdurieux.github.io

Tools affect outcomes through the defects they reveal, the decisions people make, and the way work flows. The same gate can prevent rework or create a bottleneck. Its net effect depends on precision, ownership, timing, adoption and independent validation.

Four connected lenses; an explicit cascade

The four layers are distinct, not independent. Wrong or missing upstream intent can be inherited by models, code and tests, with consequences for users and delivery. Page 3 makes propagation, shared-error blindness and containment visible. Your supplied definitions now anchor the model.



Mediated paths, not a direct tool-to-DORA conversion. Precise signs and conditions are in the graph data.

The central reinforcing loop

Effective checking → validated correction → reliable shared knowledge → fewer escaped defects → less reactive work → more capacity for prevention. The last step requires an explicit allocation decision.

The competing failure loop

Stricter controls → greater gate burden → longer waits → larger batches → more failed changes → pressure for still stricter controls. A more comprehensive gate can make the system perform worse.

The purpose and boundary

Follow one application or service from hypothesis or technical-change need through production use and learning. Optimize useful, safe outcomes and sustainable change capability. Keep suppliers, platform constraints and release policy visible.

Model contents: 20 artefact mappings · your 9 quality elements · 5 delivery metrics · 8 closed feedback loops · an explicit defect cascade, both ways · stocks, interventions and a measurement design.

What “quality” means here

Your nine quality elements, in the supplied order [S9]

Software quality concerns meeting users’ needs, behaving under real conditions and remaining maintainable over time. The meanings below reproduce your definitions; the indicators are proposed measurements.

Quality category	Operational meaning	Observe independently
Q1 Functional correctness and completeness	Performs the required tasks accurately and covers relevant business rules and edge cases.	Independent acceptance outcomes; escaped functional defects; task completion.
Q2 Reliability and resilience	Works consistently, handles failures gracefully, and recovers without losing or corrupting data.	SLO attainment, failure injection and verified data recovery.
Q3 Usability and accessibility	Is understandable, efficient to use, and accessible to people with different abilities.	Observed task success, user errors and accessibility checks.
Q4 Performance and efficiency	Responds quickly and uses computing resources appropriately.	Latency and resource use under a stated workload.
Q5 Security and privacy	Prevents unauthorized access, protects information, and handles personal data appropriately.	Verified controls, access tests, privacy requirements and exposure.
Q6 Maintainability	Is easy to understand, diagnose, fix, and change without introducing defects.	Diagnosis and change effort; change coupling; regression burden.
Q7 Testability	Allows its behaviour to be verified through effective, repeatable tests.	Repeatable independent tests, controllability and observability.
Q8 Compatibility and adaptability	Integrates with other systems and accommodates changes in platforms, requirements, or workload.	Consumer tests, migration and adaptation scenarios.
Q9 Operability	Is straightforward to deploy, monitor, support, and troubleshoot in production.	Deployment, monitoring, support and recovery evidence.

Keep four connected lenses distinct

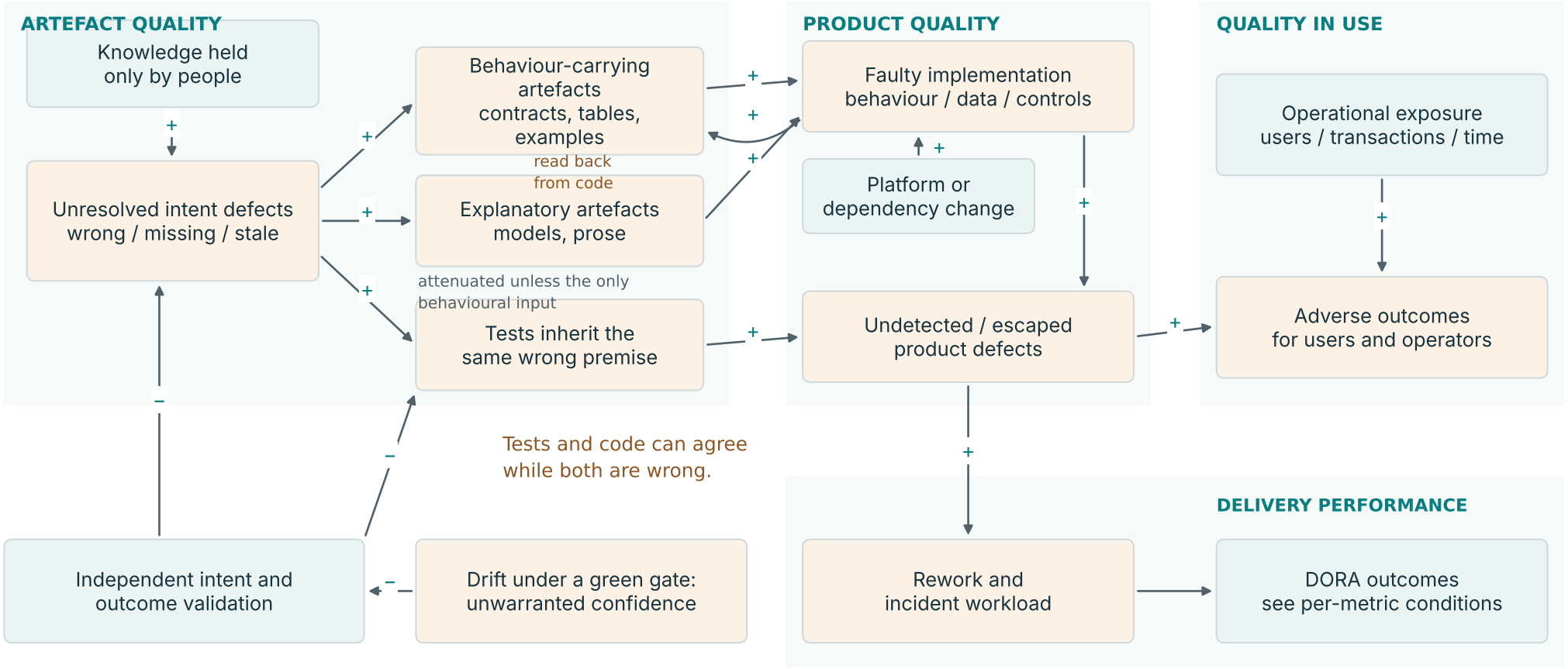
Artefact quality concerns what is written; product quality, what is built; quality in use, what people experience; DORA, how changes are delivered. Defects can cross these boundaries. Distinguishing the lenses lets us test each link instead of assuming it.

Use measurable, product-specific acceptance criteria. Technical debt is deferred work and future change burden. A gate score or delivery metric does not establish the nine quality elements.

How defects cascade

One upstream defect can be reproduced, hidden and exposed downstream

Arrows show conditional causal paths, not a mandatory lifecycle. “+” increases the named defect or consequence; “−” reduces it. Every path can be interrupted.



Amplifiers: reach · shared-oracle blindness · late discovery · exposure. Attenuator: a higher-ranked layer that still states the behaviour.

Evidence: model-to-code transfer, the reverse path from code, attenuation by precedence and a green gate over drift have bounded experimental support [S3, S12, S13]. The people-side source, the full cascade and its DORA effects are hypotheses. More downstream impact is possible, not inevitable.

How the paths reach DORA

Five current delivery metrics; all effects here are conditional

Use the same application or service over time. A causal path is a hypothesis to test; no percentage improvement or tool-specific coefficient is assumed. [S4]

Metric	Definition / direction	Proposed path and condition
DF · Deployment frequency	Production deployments per period; higher capability to deploy is desirable.	Less rework and smaller viable changes can permit more deployments. Only if demand, deployment automation and release policy allow it; do not manufacture deploys.
CLT · Change lead time	Elapsed time from a code commit to production deployment; lower is desirable.	Fewer downstream corrections and less coordination can shorten the path. An upstream gate can move work before the commit, or add waiting after it; track both.
CFR · Change fail rate	Fraction of deployments requiring immediate remedial intervention; lower is desirable.	Fewer escaped defects in covered failure classes can reduce failures. Many defects do not cause immediate deployment failure; denominators and change mix matter.
FDRT · Failed deployment recovery time	Time to recover from a failed deployment needing intervention; lower is desirable.	Current models and trace links can shorten diagnosis and change impact analysis. Depends on actual responder use, telemetry and recovery controls; fewer failures alone do not shorten this duration.
DRR · Deployment rework rate	Fraction of deployments unplanned because of production incidents; lower is desirable.	Fewer escaped defects can reduce corrective deployments. Not all rework, debt work or planned improvement qualifies; keep classification consistent.

The upstream measurement blind spot

Change lead time starts at the code commit. Earlier clarification can reduce total delivery time without moving CLT; mandatory pre-commit preparation can improve CLT while making the whole journey slower. Capture both clocks.

Track timestamps for hypothesis, first relevant code commit, production deployment, user release and validated outcome. These are event landmarks, not an assumption that discovery and delivery happen in non-overlapping phases. Deployment and release can be separate.

Additional guardrails

User task success; SLO attainment and incident impact; security and safety outcomes; performance; accessibility; debt trend; staff workload. DORA alone cannot tell whether a useful, safe product was built.

Artefact pathways 01–05

All 20 inventory rows; site states are preserved [S1]

“Covered” means that checks exist for some properties. “Planned” and “open” are not credited as deployed capability. Quality and metric codes are defined in the preceding sections.

A01 PlantUML models

COVERED

pumllint: semantic rules, score, ratchet, trace, editor and CI

Mechanism: Earlier detection of supported modelling defects; author resolves missing decisions before implementation.

Quality: Q1, Q2, Q6, Q7. CFR/DRR may fall via fewer escaped defects; CLT may fall via less rework; FDRT only if current models aid response.

Limit: A maturity score is not product correctness. Diagram-code drift is not checked by the listed gate. Author-less semantic invention can harm. Drift under a green gate is measured: one maintained set at Level 5, 100/100, with 25 claims the code contradicts (S12 F4).

A02 ARIS process models (EPC)

COVERED

aris2puml converts; pumllint checks the resulting activity diagrams

Mechanism: Expose supported process defects to executable checks without silently repairing their source meaning.

Quality: Q1, Q2, Q6. Less process misunderstanding may reduce rework and failure; CLT, CFR and DRR are indirect outcomes.

Limit: Conversion fidelity, supported constructs and author review matter. In-ARIS validation still needs an adopter; no automatic transfer of sequence-codegen effect sizes.

A03 Textual requirements

PLANNED

StrictDoc, Doorstop, Sphinx-Needs, OpenFastTrace for structure and links

Mechanism: Find missing or broken references; make requirement ownership and change impact easier to resolve.

Quality: Q1, Q6, Q7. Less clarification and omitted impact may improve CLT, CFR and DRR.

Limit: Current structural checks do not establish truth or semantic completeness. The wording linter is planned; ambiguity detection is advisory.

A04 User stories

OPEN

No maintained checker listed; proposed complement: examples and joint refinement

Mechanism: People expose missing cases and conflicting expectations before code is written.

Quality: Q1, Q3. Avoided misunderstanding may reduce total change time and corrective deployments.

Limit: This is a human-validation intervention, not a shipped machine gate. A story template can be complete while the problem is misunderstood.

A05 Specification prose

COVERED

Vale and markdownlint

Mechanism: Consistent terminology and presentation reduce avoidable reading and maintenance friction.

Quality: Q6. A weak, indirect CLT pathway through comprehension; no direct claim about runtime defect prevention.

Limit: Well-written prose may still contain vague guards or wrong business rules; style cleanup can crowd out substantive review.

Artefact pathways 06–09

All 20 inventory rows; site states are preserved [S1]

“Covered” means that checks exist for some properties. “Planned” and “open” are not credited as deployed capability. Quality and metric codes are defined in the preceding sections.

A06 Acceptance scenarios (Gherkin)

COVERED

gplint; proposed complement: execute independently reviewed examples

Mechanism: Check scenario conventions, then connect useful examples to executable acceptance evidence.

Quality: Q1, Q7. CFR and DRR can improve only through defects actually detected and fixed; CLT through faster diagnosis.

Limit: Linting Gherkin is not running tests. Tests generated from the same mistaken specification can agree with wrong code.

A07 BPMN process models

PLANNED

bpmnlint and Camunda plugins; repository-wide extension planned

Mechanism: Detect supported local process-modelling violations before implementation or process execution.

Quality: Q1, Q2, Q6. Potentially fewer process defects and less rework: CFR/DRR down and CLT down.

Limit: Do not credit the planned cross-model rules or score today. Local structural validity does not prove business behaviour.

A08 Decision tables (DMN)

COVERED

dmnlint, with the site's stated small rule set

Mechanism: Remove the specific decision-table defects the configured rules detect.

Quality: Q1, Q7. Fewer decision errors may reduce CFR/DRR, if those defects otherwise reach production.

Limit: Do not infer complete overlap, gap, legal or business-rule verification from a lint pass; use independent boundary examples. Measured: 13 tables with 0 gaps, 0 overlaps and 1 untyped output, which was the defect; output typing caught what gap analysis passed (S12 F6).

A09 Cross-artefact consistency

PLANNED

pumllint trace today; requirement-model-test-API identifier graph planned

Mechanism: Explicit identifiers expose orphan references and missed change impact across connected artefacts.

Quality: Q1, Q6, Q7, Q8, Q9. Less omitted change scope may improve CLT/CFR/DRR; trace-informed diagnosis may improve FDRT.

Limit: A valid link is not semantic agreement or test adequacy. The full graph and diagram-code drift detection are not shipped by this inventory. Measured: trace reported 158/158 verified while 33 specified statements were never executed; clause-level coverage is the gap. Deleting a requirement's only scenario trips the trace gate; deleting the rule from a requirement that keeps a scenario does not (S12 F6, S13).

Artefact pathways 10–13

All 20 inventory rows; site states are preserved [S1]

“Covered” means that checks exist for some properties. “Planned” and “open” are not credited as deployed capability. Quality and metric codes are defined in the preceding sections.

A10 Architecture decisions (ADR)

OPEN

adr-guard and adrpy-ai, both described as pre-1.0

Mechanism: Preserved rationale, ownership and decision status may reduce repeated debate and contradictory changes.

Quality: Q6, Q8, Q9. Potential CLT and FDRT improvement through comprehension and diagnosis.

Limit: File validation does not validate a trade-off; stale rationale can entrench a poor design. Keep the site's open classification.

A11 C4 architecture models

COVERED

Structurizr CLI validation and inspections

Mechanism: Make architecture structure easier to inspect and identify supported inconsistencies.

Quality: Q6, Q8, Q9. Fewer coordination mistakes may improve CLT; current maps may aid FDRT.

Limit: A valid architecture model does not prove that deployed dependencies match it or that the system meets reliability or performance goals.

A12 ArchiMate enterprise models

OPEN

Archi GUI validator and custom scripts, as listed

Mechanism: Human interpretation of validated relationships may improve cross-team change impact decisions.

Quality: Q6, Q8. Potential coordination and CLT benefit, mediated by governance decisions.

Limit: No shared gate-capable catalogue is claimed by the snapshot. Central approval can add more delay than modelling removes.

A13 SysML v2 and Capella

OPEN

Pilot implementation and Capella validation rules

Mechanism: Check supported systems-model constraints and clarify interfaces or stated requirements.

Quality: Q1, Q7, Q8. Potential integration-failure reduction; delivery effects depend on system-software interfaces and validation.

Limit: No general standalone gate is credited by the snapshot. Formal model validity is not a safety case; safety requires hazard-specific evidence.

Artefact pathways 14–17

All 20 inventory rows; site states are preserved [S1]

“Covered” means that checks exist for some properties. “Planned” and “open” are not credited as deployed capability. Quality and metric codes are defined in the preceding sections.

A14 API contracts (OpenAPI, AsyncAPI)

COVERED

Spectral, vacuum, Redocly; vacuum score and minimum

Mechanism: Enforce configured contract conventions and make interface expectations explicit before consumer implementation.

Quality: Q5, Q6, Q8. Interface defects and rework may fall, improving CFR/DRR/CLT.

Limit: A conforming document is not a verified implementation. Security benefits require relevant rules and tests; backward compatibility is not automatic.

A15 Schemas (Protobuf, GraphQL, JSON Schema)

COVERED

buf, graphql-eslint, sourcemeta jsonschema; buf breaking checks

Mechanism: Catch supported schema defects and incompatible changes before dependent consumers fail.

Quality: Q1, Q6, Q8. A relatively short causal path to fewer integration failures and corrective deployments: CFR/DRR down.

Limit: Schema compatibility does not establish semantic compatibility or safe data migration. Runtime and consumer coverage still matter.

A16 Data models

COVERED

SQLFluff, dbt-checkpoint, dbt_project_evaluator

Mechanism: Improve supported SQL/project conventions and surface maintainability or structural issues.

Quality: Q1, Q6. Less data-pipeline rework may improve CLT and DRR; some detected defects may affect CFR.

Limit: The site leaves conceptual and logical models uncovered. Lint passes do not prove data correctness, efficient queries or safe migrations.

A17 Threat models

OPEN

pytm and Threagile, with custom risk gating

Mechanism: Make trust boundaries and attack paths discussable; link threats to implemented and verified controls.

Quality: Q2, Q5. Security risk may decrease without any immediate DORA change; CFR/DRR only for affected deployment incidents.

Limit: Generating a threat report is not mitigation. Custom thresholds, control ownership and security testing are required.

Artefact pathways 18–20

All 20 inventory rows; site states are preserved [S1]

“Covered” means that checks exist for some properties. “Planned” and “open” are not credited as deployed capability. Quality and metric codes are defined in the preceding sections.

A18 Process design against reality

PLANNED

Process-mining conformance, for example PM4Py; custom gate

Mechanism: Compare observed event traces with the model, then investigate deviations and revise design or execution.

Quality: Q1, Q6, Q9. Reduced recurrent process defects and bottlenecks may improve end-to-end time, CFR and DRR.

Limit: Requires suitable logs and case identities. Deviations can be beneficial adaptations; conformance to a bad process is not quality.

A19 Vocabulary (glossary)

OPEN

No gate in the inventory; the experiment's glossary check (terms used repeatedly and defined nowhere) is a candidate

Mechanism: A defined term is one identity across artefacts; an undefined recurring term is a decision nobody wrote.

Quality: Q1, Q6. Fewer silent disagreements between artefacts may reduce rework and omitted change scope: CLT and CFR, indirect.

Limit: Measured once: 42 of 60 recurring kit terms had no definition before wave 2 (S12 F8). A definition is not agreement on meaning.

A20 Property-based invariants

COVERED

Covered by a practice: hypothesis-style property tests on the specified invariants, run against the original and each rebuild

Mechanism: Properties exercise behaviour no example scenario reaches, independently of the examples the kit states.

Quality: Q1, Q2, Q7. Detection of behaviour the scenario suite misses: CFR and DRR, indirect.

Limit: Measured once: six intrinsic properties at 100 examples each and one differential property, 7/7, on one program (S12). A property holds for the cases generated, not for all.

Coverage is uneven; tool effects are not additive

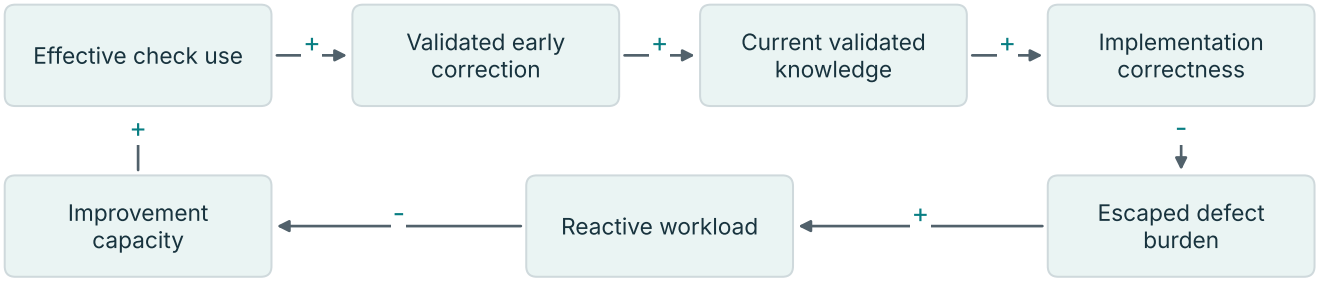
The clearest proposed paths concern correctness, compatibility and maintainability. Performance, usability, security and other properties need their own verification. Overlapping tools do not create additive benefits; shared assumptions can create common-mode failure.

Feedback that compounds

R = reinforcing: it can amplify improvement or deterioration

+ means same direction; − means opposite direction, with other factors held comparable. The loop sign comes from the product of its link signs. Delays and edge conditions are explicit in the JSON model.

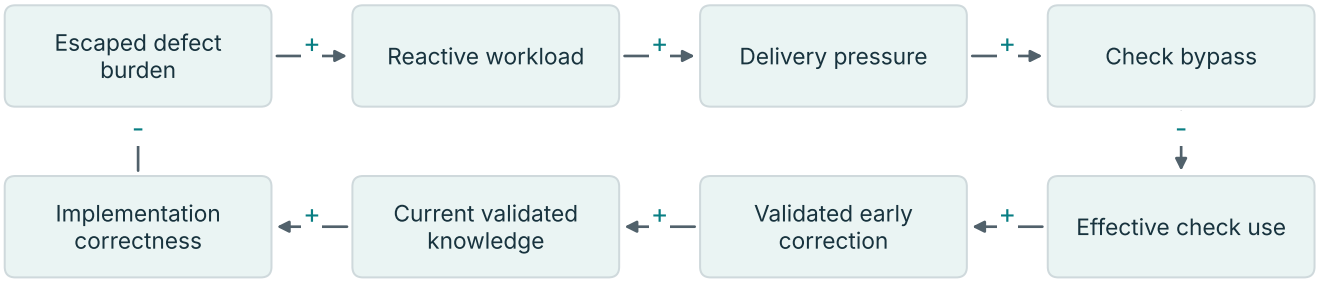
R1 Prevention and reinvestment



Better effective checking can reduce reactive work and release capacity for more prevention. It can also run in reverse. Protect some freed capacity for authorship, rule tuning and debt reduction.

Evidence: H as a whole; E03 has bounded experiment and related research support.

R2 Firefighting and bypass



Escapes consume time, pressure rises, checks are bypassed and the defect-prevention system deteriorates. Limit WIP and protect time to resolve findings; reward surfaced uncertainty.

Evidence: H.

R3 Approval and batching trap



Responding to failure with more burdensome approval can create larger batches and further failure. Use fast, risk-sensitive checks and peer decisions close to the work; monitor queues.

Evidence: R-related capability evidence; H for a particular local implementation.

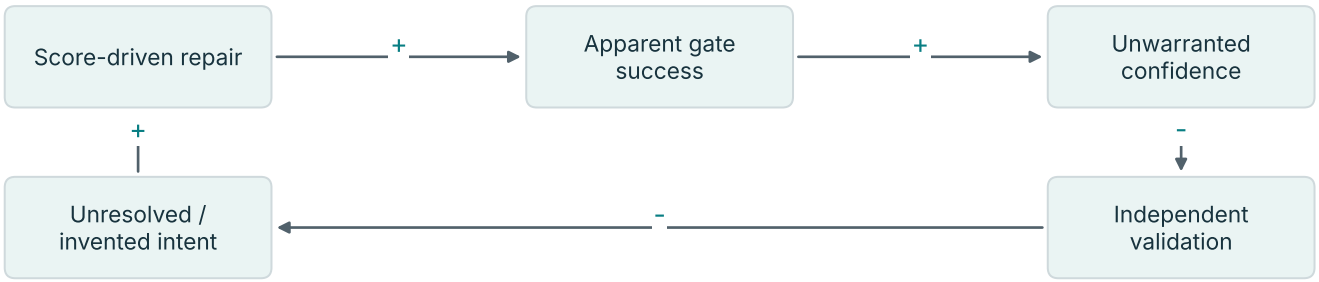
Reinvestment is a decision: saved time can instead become extra demand. That rebound increases WIP and waiting, eroding the intended benefit.

False assurance and correction

R4 is a failure loop; B1 and B2 counter a discrepancy

The existence of a balancing loop does not guarantee a good outcome. Delay, weak evidence, overload or an incorrect target can prevent effective correction.

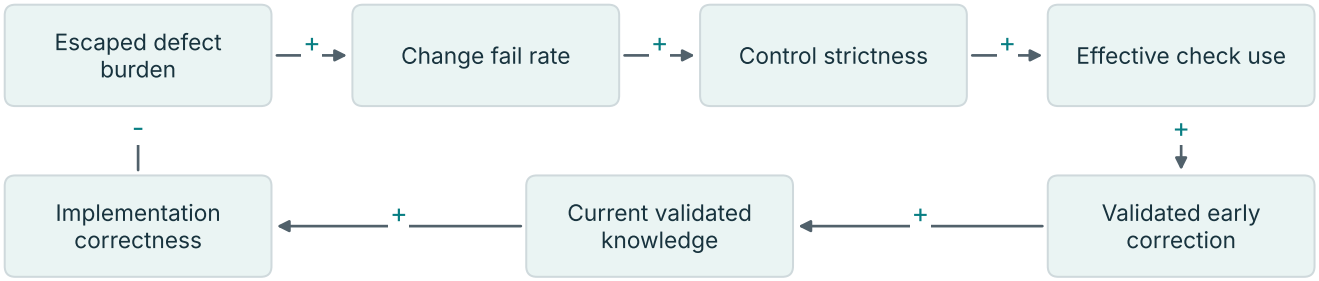
R4 Green but wrong



Score-oriented semantic invention can produce apparent success, reduce independent challenge and perpetuate unresolved intent. Escalate missing intent to its owner; require independent outcome examples, not just a higher score.

Evidence: M for E27 and bounded repair; M-bounded for drift feeding confidence (E77); H for the organizational loop.

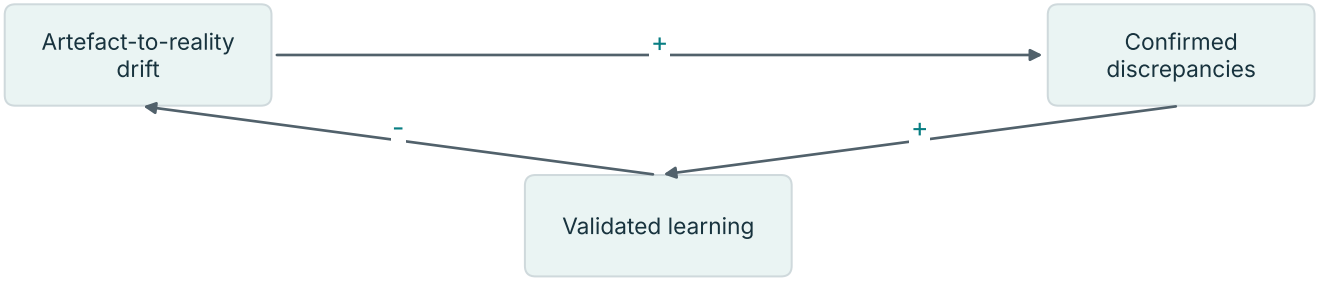
B1 Corrective risk control



Relevant controls can counter deployment failures, provided they improve effective checking instead of merely adding burden. Compare B1 against competing R3; calibrate precision and response, not strictness alone.

Evidence: H.

B2 Reality-based learning



Observation exposes mismatch, and owned learning reduces it by changing the model or the implementation. Maintain useful runtime evidence and independent review; conformance is not automatically the right objective.

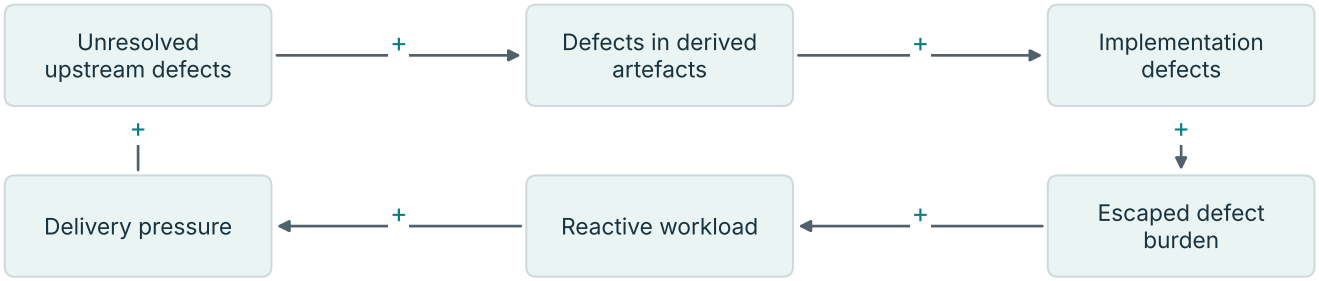
Evidence: H; one instance M-bounded: a model set read against its code, adjudicated, spot-checked by execution (S12 F4).

Why downstream effects can grow

Propagation and amplification are different mechanisms

The root error need not become more wrong. Its reach, persistence, repair burden or consequences can increase. Independent evidence, containment and early correction can attenuate the cascade.

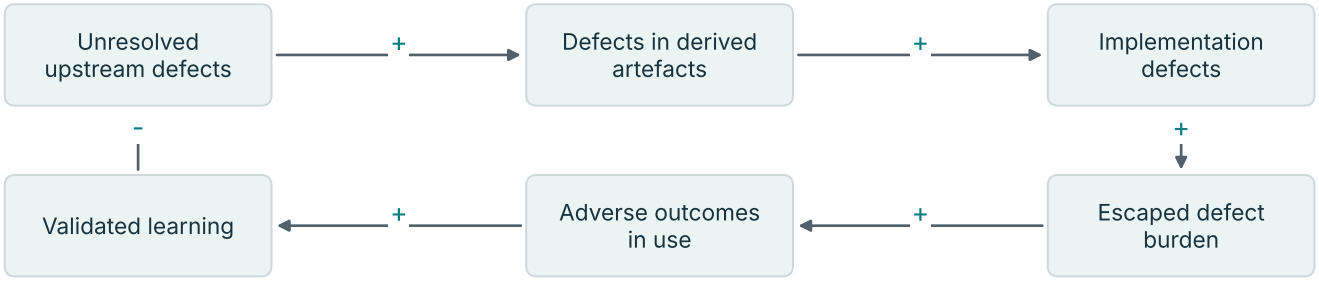
R5 Cascade and rushed repair



Unresolved intent propagates into implementation; escaped defects generate reactive work and pressure, which can encourage further unresolved intent. Resolve the root assumption and all affected dependants; protect time for validation rather than just patching symptoms.

Evidence: H as a loop; the model-to-code leg has bounded M support.

B3 Outcome-led source correction



Observed harm can drive validated learning that corrects the originating intent and interrupts recurrence. Use independent user and operational evidence, accountable source owners and impact-aware correction.

Evidence: H; depends on observation, attribution and action.

What “worse downstream” can mean

Spread: more dependent artefacts and components carry consequences. Concealment: shared assumptions delay discovery. Entrenchment: more decisions, data and commitments must be unwound. Exposure: more users or transactions experience the fault. These are separate variables, not a universal cost multiplier.

What to measure

Link each confirmed root defect to the downstream items actually affected. Record detection stage and delay, escapes, independent checks that stopped it, repair scope and effort, and user impact. Do not count every copy as an independent root cause.

Stocks, delays and bottlenecks

A dynamic structure, without invented numerical coefficients

Stock	Additions / removals	What the measure does not mean
U · Unresolved upstream defects **Count within an explicit artefact scope**	introduced by new or changed intent; removed by validated correction	A warning count is an observation, not the stock itself.
T · Technical and information debt **Estimated remediation effort, hours**	deferred work and drift add effort; verified remediation removes it	Estimates require re-estimation; not interchangeable with defect count.
K · Validated, current shared knowledge **Count of owned, validated knowledge items**	validation adds; invalidation and stale context remove	Coverage, relevance and use matter more than volume.
W · Work in progress and waiting **Changes in a consistently defined workflow**	admitted changes add; completed or abandoned changes remove	A larger W stock can increase waiting even if coding is faster.
N · Unresolved or invented decisions **Assumption-register entries per artefact set**	unstated intent adds; validation and externalisation remove	Measured on five rebuilds: 65, 50, 30 from a code-derived kit; 117, 94 from a generated wiki. A count of assumptions, not of defects.
P · Knowledge held only by people **Items in a capture inventory: found, validated, externalised with a check, residual by name**	elicitation and externalisation remove; departures and unrecorded decisions add	No percentage of knowledge captured: the population has no denominator. Its downstream shadow is N.

Conservation statements

U(next) = U + introduced defects − validated corrections.
T(next) = T + newly deferred effort + re-estimated drift burden − retired effort.
K(next) = K + validated knowledge − invalidated or stale knowledge.
W(next) = W + admitted changes − completed changes − abandoned changes.

Use consistent units and periods. A future simulation needs locally estimated rates, delays and capacity constraints, with stocks bounded at zero. These are bookkeeping structures, not fitted equations.

Delays and bottlenecks

Time bands and saturation are assumptions to check locally

Three relevant delays

Minutes to a change: check feedback, triage and author corrections. Days to weeks: altered implementation and escaped-defect observations. Weeks to months: confidence, skills, debt repayment and rule maintenance. These time bands are assumptions to check locally.

Why the effect is nonlinear

Coverage saturates: the next rule can add little value. Low precision consumes scarce attention. As demand approaches available review capacity, waiting can rise sharply. Better upstream information may not improve delivery if release policy or deployment infrastructure remains the bottleneck.

People, power and lifecycle

Tools are actants; people retain decisions and accountability

Actor	Decision / leverage	Needed feedback or safeguard
Users and domain owners	Validate intent, edge cases and acceptable outcomes	Can reject a semantically wrong 'green' artefact; supply independent examples.
Product owners and sponsors	Choose scope, priorities and improvement capacity	Do not convert freed capacity automatically into more unfinished work.
Analysts and architects	Own models, interfaces, rationale and dependencies	Maintain source truth; distinguish unsupported constructs and missing decisions.
Developers and AI agents	Consume artefacts and implement changes	Resolve uncertainty with an owner; agents cannot authorize invented business meaning.
QA, security and safety specialists	Choose independent acceptance and risk evidence	Challenge shared assumptions; gate on justified risk, not document appearance.
Platform and gate maintainers	Own check precision, latency, versions and integration	Tune rules, make failures actionable, audit suppressions and baseline changes.
Operations and responders	Use current context and report production behaviour	Feed incident causes and observed drift back to authors; verify recovery paths.
Governance and risk owners	Set proportional controls and exceptions	Time-bound waivers with accountable ownership; change incentives when scores are gamed.

Anchor the causal network in the SAFe delivery wheel

Continuous Exploration: Hypothesize; Collaborate & Research; Architect; Synthesize.
Continuous Integration: Develop; Build; Test End-to-End; Stage.
Continuous Deployment: Deploy; Verify; Monitor; Respond.
Release on Demand: Release; Stabilize; Measure; Learn.

The inventory acts mainly on exploration and integration. Deployment and release outcomes are mediated by runtime practices. This model adds the return path from user and operational evidence to upstream decisions; it does not extend the tool catalogue’s coverage by assertion. [S1]

A change in incentives is a stronger intervention than a score target

Reward resolution of real uncertainty and verified outcomes. Do not rank individuals by DORA, finding count, diagram maturity or number of links. Allow teams to challenge a bad rule and make exceptions visible, owned and temporary.

Interventions that change the loops

Use the six gate-contract properties as a system, with intent validation

P1 Run at the author and in CI

Shorter feedback delay and less avoidable handoff work. Measure time-to-action and usefulness, not tool-install count; S1/S6.

P2 Separate clean / findings / unable-to-run

Makes the check result actionable and its failure visible. An unexecuted check must not silently become a pass; S1.

P3 Use severity and a justified threshold

Focus scarce review effort on consequential defects. Do not hard-block low-precision ambiguity judgements; validate rule precision locally; S1.

P4 Baseline and ratchet

Allows brownfield adoption while exposing regression. Baseline update outside the gate; new/untracked work needs an explicit policy. A ratchet alone does not repay old debt; S1/S3.

P5 Use stable IDs and machine-readable reports

Makes change-impact links and review evidence retrievable. Record what inputs, versions and scope were checked; a link or report is not a semantic proof; S1.

P6 Tag evidence and publish failures

Limits false confidence and supports revision of rules. Keep local experiment results separate from external research and untested mechanisms; S1/S3.

P7 Validate intent with people and independent tests

Prevents common-mode errors across requirements, models, tests and code. Site-level complement to the inventory: stakeholders, walking skeleton and shipped-product feedback; S1/S10.

P8 Review incidents and observed deviations

Closes the loop from actual outcomes to upstream intent. Proposed implementation practice; distinguish useful exceptions from defects and update artefacts deliberately; H.

Interventions, continued

Four practices added in v0.3: one measured, three from the departing-expert plan

P9 Calibrate the oracle by mutation

Tests the tests before any rebuild, so detection is independent rather than shared-premise. Planted mutants must fail the suite; m1 failed 2 scenarios and m2 failed 11 before any recreation ran (M-bounded, S12 F7).

P10 Elicit what only people know

Fills upstream omissions at their source and shrinks the assumption stock. Inventory from traces before any session; worked cases, counterexamples, exception walks, incident replay, quirk and orphan adjudication, with the successor present (H, S14).

P11 Externalise into the normative home, with a check

Turns an elicited item into shared, current knowledge that drift can be measured against. Rules to decision tables, terms to the glossary, flows to models, reasons to decision records, procedures to runbooks; the register links provenance and is never a second source of truth (H, S14).

P12 Validate every extracted claim with its owner

The independent channel that breaks the reverse path from code into derived artefacts (E73). Observed, inferred and intended kept apart; a defect preserved for compatibility is a recorded decision with scope and an example; feeds E57 (H as a practice; the channel is M-bounded, S12 F2).

Suggested order of leverage

1. Fix the purpose: useful, safe outcomes rather than passing scores.
2. Clarify decision rights: who resolves missing intent and accepts residual risk.
3. Shorten information feedback: author-local checks and usable incident evidence.
4. Adjust flow and rules: precision, thresholds, ratchets and WIP limits.
5. Add tool coverage only where an observed defect class justifies it.

This is a design proposal, not an empirical ranking. Co-design with authors, users, responders and gate maintainers. Their perspectives and power differ; participation is proposed here, not claimed to have occurred. Reframe the model as experiments reveal new dynamics.

Make the model testable

A small service-level pilot; no assumed DORA effect size

H1 Precise early checks prevent relevant escaped defects

Run an applicable high-precision rule subset; record confirmed findings, corrections and targeted escape classes.

Prediction: Confirmed early catches rise initially; targeted escapes and unplanned correction work fall later.

Challenge: If escapes do not change despite genuine coverage and consumption, inspect rule relevance, propagation and test blind spots.

H2 Gate burden can outweigh prevention

Track wait and triage time by stage; introduce advisory-first checks and compare similar change cohorts.

Prediction: False positives or centralized sign-off increase waiting and bypass before any delayed quality gain.

Challenge: If burden is material without prevented defect evidence, tune, relocate or remove the gate instead of escalating strictness.

H3 Traceability reduces omitted change impact

For changes with known dependent consumers, compare identified impacts with independently reviewed actual impacts.

Prediction: Fewer missed consumers and shorter diagnosis after the trace is used.

Challenge: If links exist but are stale or unused, graph coverage is not a leading indicator of the claimed benefit. Measured once: trace reported 158/158 requirements verified while 33 specified statements were never executed (S12 F6).

H4 Intent validation prevents 'green but wrong' repairs

Flag guessed or unresolved decisions; require domain-owner resolution and independent outcome examples.

Prediction: Incorrect repairs decline even if gate scores change little.

Challenge: If no independent oracle exists, do not infer correctness from matching specifications and tests. Measured once: a kit derived from code wrote the original's crash into the spec as a requirement, the rebuild reproduced it, and the suite scored it as correct (S12 F2).

Prospective use: choose one causal path

Pick one service, one real defect class and the applicable artefacts. Establish observation before enforcing a gate. Agree check scope, finding review, time budget and independent correctness evidence. Run advisory first, then enforce only validated high-precision rules.

Diagnostic use: work backward from the outcome

If CLT worsens, locate the waiting stage before adding rules. If incidents persist despite green artefacts, inspect semantic validity, coverage, drift and downstream use. If FDRT is unchanged, check whether responders use the model and whether diagnosis is the limiting step.

Suggested cadence: a baseline window, then a staged pilot and follow-up spanning several delivery cycles. Choose duration from deployment frequency and event counts; a fixed calendar window cannot guarantee enough failures to infer an effect.

Measure mechanisms before outcomes

A falsifiable learning plan, with evidence limits attached

Record	Why it matters
Relevant check exposure	Eligible artefacts and changes; rule/version/config; whether run and consumed. Separate tool installation from effective use.
Finding usefulness	Independently adjudicated actionable findings; samples of missed defects; duplicate/noise rate; time to action. Findings rising initially can mean better detection.
Costs and behaviour	Triage hours, execution delay, approval wait, bypasses, suppressions, baseline updates, batch size, WIP and reactive work share.
Downstream outcomes	Covered defect escape classes, independent acceptance, user/SLO/security outcomes, debt trend, all five DORA metrics and both delivery clocks.
Context and provenance	Service, change ID, risk/size, team and platform changes, deployment and incident IDs, artefact versions, authorship and unresolved decisions.

Evaluate direction and trade-offs, not a vanity score

Define acceptable gate burden and the targeted quality improvement before enforcement. Compare similar change cohorts or use a phased introduction with a credible comparison service. Inspect pre-existing trends and changes in staffing, demand, release policy, test coverage and incident reporting.

Show deployment counts and failure/rework counts alongside rates. For CLT and recovery, show distributions and event counts; fewer incidents can change which failures remain. If no failed deployments occur, report no observed recovery-time sample, not zero recovery time. No single-service before/after result proves causality.

What is actually measured in pumllint

The author-less repair experiment raised the low-maturity tier’s gate results while executed pass rate fell 5.9 percentage points relative to its degraded baseline. This supports a specific “green but wrong” failure mode on that instrument. It does not estimate a DORA effect or establish the same effect for every artefact class. [S3]

What the out-of-sample test measured

v0.3: the review and the degradation test, both in pumllint

What the out-of-sample test measured (v0.3)

A kit derived from code alone let independent agents rebuild a 3 500-line program cell for cell; the same route copied the code's defects into the kit, and a suite calibrated on the code scored the inheritance as correct. Fifteen defects planted in the kit, one layer at a time: six reached the code, all six passed the kit's own suite and failed the independent one, every outcome following the declared precedence of the layers. [S12, S13]

Evidence status of this whole model

M: bounded local experiment. R: related primary capability research. H: proposed causal mechanism. I: site inventory. All tool-to-production-DORA paths remain hypotheses here; no production intervention dataset was supplied.

Sources and model maintenance

Readable report plus a machine-readable causal graph

S1 Legible site and artefact inventory

github.com/fdurieux/fdurieux.github.io/blob/9ba9b09/index.html
18-row inventory, six gate properties, boundaries; snapshot dated 2026-10-26

S2 Legible programme roadmap

github.com/fdurieux/fdurieux.github.io/blob/9ba9b09/ROADMAP.md
Shipping, planned and open work; programme gate contract

S3 pumllint EVIDENCE.md

github.com/fdurieux/pumllint/blob/9d728c8/EVIDENCE.md
Sequence-diagram code-generation experiments and author-less repair limitations; no production DORA effect measured

S4 DORA software delivery performance metrics

dora.dev/guides/dora-metrics/
Current five metrics and application/service measurement boundary; accessed 2026-10-28

S5 DORA: documentation quality

dora.dev/capabilities/documentation-quality/
Documentation and technical-capability research, not tool-specific intervention proof

S6 DORA: continuous integration

dora.dev/capabilities/continuous-integration/
Fast feedback, frequent integration and automated tests

S7 DORA: working in small batches

dora.dev/capabilities/working-in-small-batches/
Batch size and feedback; capability research

S8 DORA: streamlining change approval

dora.dev/capabilities/streamlining-change-approval/
Heavy approval processes, delays, batching and risk

S9 User-supplied software quality and systemic-design definitions

github.com/fdurieux/fdurieux.github.io/blob/main/model/task-brief.md
Full texts supplied on 2026-10-28 and preserved in the task brief; authoritative definitions

Sources, continued

S10 to S15; S12 and S13 are the v0.3 evidence pin

S10 DORA: test automation

dora.dev/capabilities/test-automation/
Maintainable tests of actual user journeys; checks alone do not replace test execution

S11 NASA: requirements management

www.nasa.gov/reference/6-2-requirements-management/
Requirements dependencies, impact analysis and later change cost; engineering guidance, not a measured universal amplification factor

S12 pumllint: the systemic model reviewed against the recreation experiment

github.com/fdurieux/pumllint/blob/eab25fd/recreate_experiment/legible-model-review-2026-10-28/REVIEW.md
Out-of-sample test of v0.2, artefact-to-product leg only; findings F1-F9; verdict fit for its stated purpose after specified corrections, for the cascade

S13 pumllint: degrading the kit, pre-registered and graded

github.com/fdurieux/pumllint/blob/eab25fd/recreate_experiment/degradation-2026-10-28/PREREGISTRATION.md
Fifteen defects planted in a specification kit, one layer at a time; six arms; six of fifteen reached the code, all six concealed by the arm's own suite; every outcome follows the kit's declared precedence

S14 pumllint: knowledge transfer for a departing expert

github.com/fdurieux/pumllint/blob/eab25fd/recreate_experiment/knowledge-transfer/DOMINIQUE-CASE.md
A plan, not a run: inventory from traces, elicitation, externalisation, a grading test with a denominator, a residual by name; the source of practices P10-P12

S15 pumllint: what the kit does not reach

github.com/fdurieux/pumllint/blob/eab25fd/recreate_experiment/results/coverage/UNREALISED.md
89 % of one real program's statements exercised by the kit's suite; 141 never reached; 61 with no caller, the measured size of the tacit remainder for that program

Source discipline

The tool inventory is the site’s dated snapshot, not a new exhaustive tool audit. The site table labels diagram-code drift “planned”, whereas the programme roadmap records it as “open”; this model treats it as unavailable and does not assume that either status means implemented.